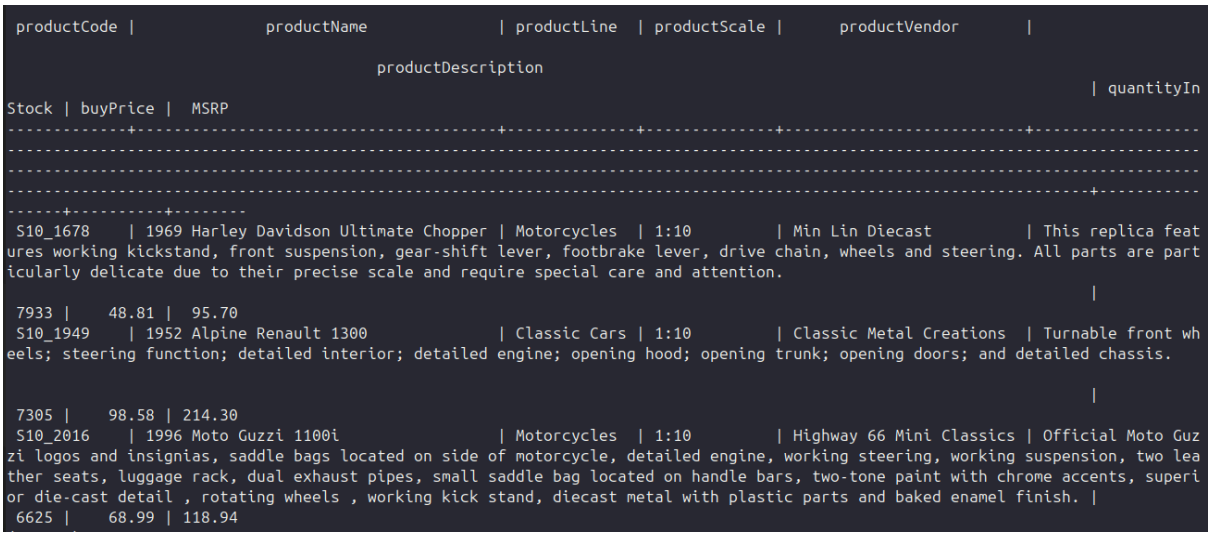
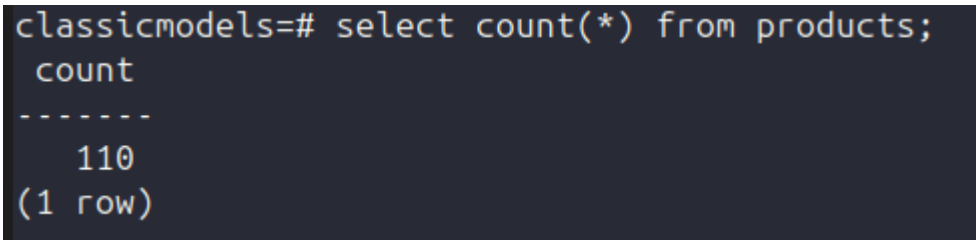
1. List all products

SELECT * FROM products;



```
 productCode |             productName             | productLine  | productScale |      productVendor      |
                                     productDescription
                                                                                                                  | quantityIn
Stock | buyPrice |  MSRP
-------------+-------------------------------------+--------------+--------------+-------------------------+-----------------
-----------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------------+-----------
------+----------+--------
 S10_1678    | 1969 Harley Davidson Ultimate Chopper | Motorcycles  | 1:10         | Min Lin Diecast         | This replica feat
ures working kickstand, front suspension, gear-shift lever, footbrake lever, drive chain, wheels and steering. All parts are part
icularly delicate due to their precise scale and require special care and attention.
                                                                                                                  |
 7933 |    48.81 |  95.70
 S10_1949    | 1952 Alpine Renault 1300              | Classic Cars | 1:10         | Classic Metal Creations  | Turnable front wh
eels; steering function; detailed interior; detailed engine; opening hood; opening trunk; opening doors; and detailed chassis.

                                                                                                                  |
 7305 |    98.58 | 214.30
 S10_2016    | 1996 Moto Guzzi 1100i                 | Motorcycles  | 1:10         | Highway 66 Mini Classics | Official Moto Guz
zi logos and insignias, saddle bags located on side of motorcycle, detailed engine, working steering, working suspension, two lea
ther seats, luggage rack, dual exhaust pipes, small saddle bag located on handle bars, two-tone paint with chrome accents, superi
or die-cast detail , rotating wheels , working kick stand, diecast metal with plastic parts and baked enamel finish. |
 6625 |    68.99 | 118.94
```

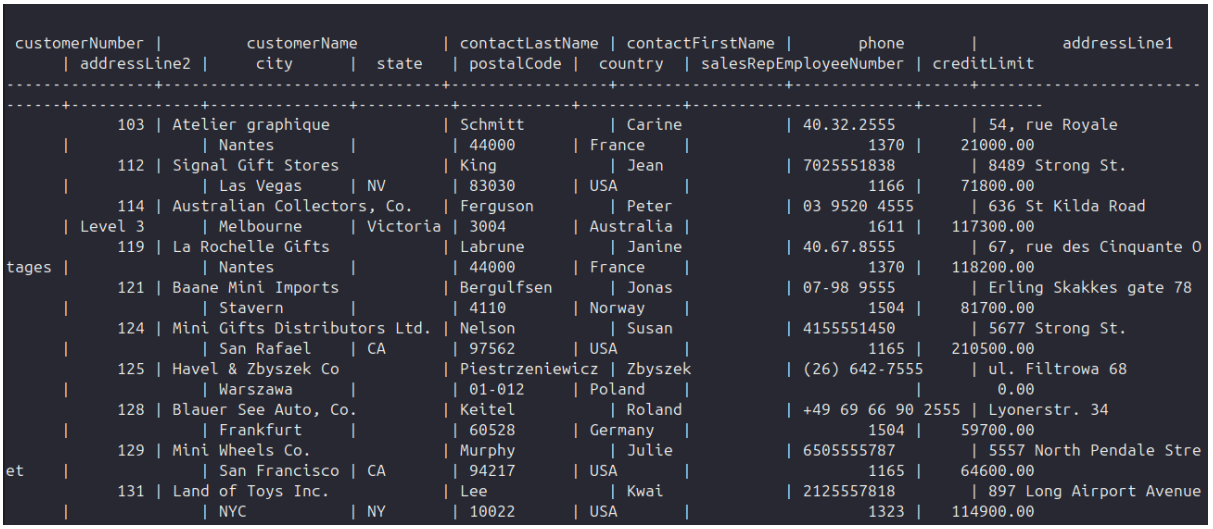


```
classicmodels=# select count(*) from products;
 count
-------
   110
(1 row)
```

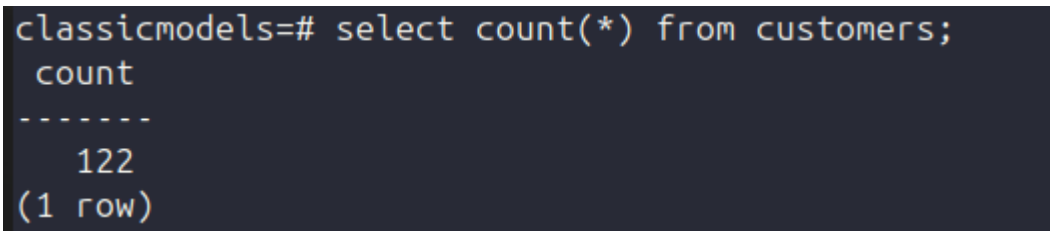
SELECT * means "give me every column" and FROM products specifies the table. PostgreSQL reads all rows from the products table and returns them AS it is. The result has 110 rows because there are 110 products in the database.

2. Get all customers

SELECT * FROM customers;



```
 customerNumber |         customerName         | contactLastName | contactFirstName |       phone       |         addressLine1
      | addressLine2 |     city      |  state   | postalCode |  country  | salesRepEmployeeNumber | creditLimit
----------------+------------------------------+-----------------+------------------+-------------------+----------------------
------+-------------+---------------+----------+------------+-----------+------------------------+-------------
           103 | Atelier graphique            | Schmitt         | Carine           | 40.32.2555        | 54, rue Royale
      |             | Nantes        |          | 44000      | France    |                   1370 |    21000.00
           112 | Signal Gift Stores           | King            | Jean             | 7025551838        | 8489 Strong St.
      |             | Las Vegas     | NV       | 83030      | USA       |                   1166 |    71800.00
           114 | Australian Collectors, Co.   | Ferguson        | Peter            | 03 9520 4555      | 636 St Kilda Road
      | Level 3     | Melbourne     | Victoria | 3004       | Australia |                   1611 |   117300.00
           119 | La Rochelle Gifts            | Labrune         | Janine           | 40.67.8555        | 67, rue des Cinquante O
tages |             | Nantes        |          | 44000      | France    |                   1370 |   118200.00
           121 | Baane Mini Imports           | Bergulfsen      | Jonas            | 07-98 9555        | Erling Skakkes gate 78
      |             | Stavern       |          | 4110       | Norway    |                   1504 |    81700.00
           124 | Mini Gifts Distributors Ltd. | Nelson          | Susan            | 4155551450        | 5677 Strong St.
      |             | San Rafael    | CA       | 97562      | USA       |                   1165 |   210500.00
           125 | Havel & Zbyszek Co           | Piestrzeniewicz | Zbyszek          | (26) 642-7555     | ul. Filtrowa 68
      |             | Warszawa      |          | 01-012     | Poland    |                        |        0.00
           128 | Blauer See Auto, Co.         | Keitel          | Roland           | +49 69 66 90 2555 | Lyonerstr. 34
      |             | Frankfurt     |          | 60528      | Germany   |                   1504 |    59700.00
           129 | Mini Wheels Co.              | Murphy          | Julie            | 6505555787        | 5557 North Pendale Stre
et    |             | San Francisco | CA       | 94217      | USA       |                   1165 |    64600.00
           131 | Land of Toys Inc.            | Lee             | Kwai             | 2125557818        | 897 Long Airport Avenue
      |             | NYC           | NY       | 10022      | USA       |                   1323 |   114900.00
```



```
classicmodels=# select count(*) from customers;
 count
-------
   122
(1 row)
```

PostgreSQL reads all rows from the customers table and returns them AS it is. The result has 122 rows because there are 122 customers in the database. Each row contains all customer fields: number, name, contact, address, phone, city, state, postal code, country, credit limit, and their assigned sales rep number.

3. Show all orders

SELECT * FROM orders;

```
 orderNumber | orderDate  | requiredDate | shippedDate | status  |                              comments
        | customerNumber
-------------+------------+--------------+-------------+---------+----------------------------------------------------------------
---------+---------------
       10100 | 2003-01-06 | 2003-01-13   | 2003-01-10  | Shipped |
         |            363
       10101 | 2003-01-09 | 2003-01-18   | 2003-01-11  | Shipped | Check on availability.
         |            128
       10102 | 2003-01-10 | 2003-01-18   | 2003-01-14  | Shipped |
         |            181
       10103 | 2003-01-29 | 2003-02-07   | 2003-02-02  | Shipped |
         |            121
       10104 | 2003-01-31 | 2003-02-09   | 2003-02-01  | Shipped |
         |            141
       10105 | 2003-02-11 | 2003-02-21   | 2003-02-12  | Shipped |
         |            145
       10106 | 2003-02-17 | 2003-02-24   | 2003-02-21  | Shipped |
         |            278
       10107 | 2003-02-24 | 2003-03-03   | 2003-02-26  | Shipped | Difficult to negotiate with customer. We need more marketing m
aterials |            131
       10108 | 2003-03-03 | 2003-03-12   | 2003-03-08  | Shipped |
         |            385
       10109 | 2003-03-10 | 2003-03-19   | 2003-03-11  | Shipped | Customer requested that FedEx Ground is used for this shipping
         |            486
```

```
classicmodels=# select count(*) from orders;
 count
-------
   326
(1 row)
```

Full scan of the orders table returning all 326 order records. Each row represents one order with its orderNumber, orderDate, requiredDate, shippedDate, status, and the customerNumber of the customer who placed it. Note that customerNumber here is a foreign key which references the customers table.

4. List all employees

SELECT * FROM employees;

employeeNumber	lastName	firstName	extension	email	officeCode	reportsTo	jobTitle
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1		President
1056	Patterson	Mary	x4611	mpatterso@classicmodelcars.com	1	1002	VP Sales
1076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1	1002	VP Marketing
1088	Patterson	William	x4871	wpatterson@classicmodelcars.com	6	1056	Sales Manager (APAC)
1102	Bondur	Gerard	x5408	gbondur@classicmodelcars.com	4	1056	Sale Manager (EMEA)
1143	Bow	Anthony	x5428	abow@classicmodelcars.com	1	1056	Sales Manager (NA)
1165	Jennings	Leslie	x3291	ljennings@classicmodelcars.com	1	1143	Sales Rep
1166	Thompson	Leslie	x4065	lthompson@classicmodelcars.com	1	1143	Sales Rep
1188	Firrelli	Julie	x2173	jfirrelli@classicmodelcars.com	2	1143	Sales Rep
1216	Patterson	Steve	x4334	spatterson@classicmodelcars.com	2	1143	Sales Rep

```
classicmodels=# select count(*) from employees;
 count
-------
    23
(1 row)
```

Returns all 23 rows from the employees table. Each employee record contains their employeeNumber (primary key), firstName, lastName, extension, email, officeCode (foreign key to offices), reportsTo (foreign key to another employee, their manager), and jobTitle.

5. Get all offices

SELECT * FROM offices;

```
 officeCode |     city      |      phone       |      addressLine1       | addressLine2 |   state    |  country  | postalCode | t
erritory
------------+---------------+------------------+-------------------------+--------------+------------+-----------+------------+--
---------
 1          | San Francisco | +1 650 219 4782  | 100 Market Street       | Suite 300    | CA         | USA       | 94080      | N
A
 2          | Boston        | +1 215 837 0825  | 1550 Court Place        | Suite 102    | MA         | USA       | 02107      | N
A
 3          | NYC           | +1 212 555 3000  | 523 East 53rd Street    | apt. 5A      | NY         | USA       | 10022      | N
A
 4          | Paris         | +33 14 723 4404  | 43 Rue Jouffroy Dabbans |              |            | France    | 75017      | E
MEA
 5          | Tokyo         | +81 33 224 5000  | 4-1 Kioicho             |              | Chiyoda-Ku | Japan     | 102-8578   | J
apan
 6          | Sydney        | +61 2 9264 2451  | 5-11 Wentworth Avenue   | Floor #2     |            | Australia | NSW 2010   | A
PAC
 7          | London        | +44 20 7877 2041 | 25 Old Broad Street     | Level 7      |            | UK        | EC2N 1HN   | E
MEA
(7 rows)
```

```
classicmodels=# select count(*) from offices;
 count
-------
     7
(1 row)
```

Returns all 7 rows from the offices table. Each row is a company office location with its officeCode, city, phone, address lines, state, country, postalCode, and territory. This is a simple full table read.

6. Show all product lines

SELECT * FROM productlines;

```
 productLine  |
                                                                                                        textDescrip
tion
                                                                                            | htmlDescription | im
age
--------------+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------------------------
------------------------------------------------------------------------------------------------+-----------------+---
----
 Classic Cars | Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you are looking for classic
muscle cars, dream sports cars or movie-inspired miniatures, you will find great choices in this category. These replicas feature
 superb attention to detail and craftsmanship and offer features such as working steering system, opening forward compartment, op
ening rear trunk with removable spare wheel, 4-wheel independent spring suspension, and so on. The models range in size from 1:10
 to 1:24 scale and include numerous limited edition and several out-of-production vehicles. All models include a certificate of a
uthenticity from their manufacturers and come fully assembled and ready for display in the home or office. |                 |
 Motorcycles  | Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harle
y Davidson, Ducati and Vespa. Models contain stunning details such as official logos, rotating wheels, working kickstand, front s
uspension, gear-shift lever, footbrake lever, and drive chain. Materials used include diecast and plastic. The models range in si
ze from 1:10 to 1:50 scale and include numerous limited edition and several out-of-production vehicles. All models come fully ass
embled and ready for display in the home or office. Most include a certificate of authenticity.
                                                                                                            |                 |
 Planes       | Unique, diecast airplane and helicopter replicas suitable for collections, as well as home, office or classroom d
ecorations. Models contain stunning details such as official logos and insignias, rotating jet engines and propellers, retractabl
e wheels, and so on. Most come fully assembled and with a certificate of authenticity from their manufacturers.
```

```
classicmodels=# select count(*) from productlines;
 count
-------
     7
(1 row)
```

Returns all 7 product line categories from the productlines table. Each row has a productLine name (the primary key), a textDescription (plain text), an htmlDescription (HTML formatted text), and an image field. This is a reference/lookup table that the products table references via its productLine foreign key.

7. List all payments

SELECT * FROM payments;

customerNumber	checkNumber	paymentDate	amount
103	HQ336336	2004-10-19	6066.78
103	JM555205	2003-06-05	14571.44
103	OM314933	2004-12-18	1676.14
112	BO864823	2004-12-17	14191.12
112	HQ55022	2003-06-06	32641.98
112	ND748579	2004-08-20	33347.88
114	GG31455	2003-05-20	45864.03
114	MA765515	2004-12-15	82261.22
114	NP603840	2003-05-31	7565.08
114	NR27552	2004-03-10	44894.74
119	DB933704	2004-11-14	19501.82
119	LN373447	2004-08-08	47924.19
119	NG94694	2005-02-22	49523.67
121	DB889831	2003-02-16	50218.95
121	FD317790	2003-10-28	1491.38

```
classicmodels=# select count(*) from payments;
 count
-------
   273
(1 row)
```

Returns all 273 payment records. The payments table has a composite primary key (customerNumber, checkNumber) together uniquely identifying each payment. Each row also has paymentDate and amount..

8. Get product names and prices

SELECT "productName", "buyPrice", "MSRP" FROM products;

productName	buyPrice	MSRP
1969 Harley Davidson Ultimate Chopper	48.81	95.70
1952 Alpine Renault 1300	98.58	214.30
1996 Moto Guzzi 1100i	68.99	118.94
2003 Harley-Davidson Eagle Drag Bike	91.02	193.66
1972 Alfa Romeo GTA	85.68	136.00
1962 LanciaA Delta 16V	103.42	147.74
1968 Ford Mustang	95.34	194.57
2001 Ferrari Enzo	95.59	207.80
1958 Setra Bus	77.90	136.67
2002 Suzuki XREO	66.27	150.62
1969 Corvair Monza	89.14	151.08
1968 Dodge Charger	75.16	117.44
1969 Ford Falcon	83.05	173.02
1970 Plymouth Hemi Cuda	31.92	79.80
1957 Chevy Pickup	55.70	118.50

```
classicmodels=# select count(*) from products;
 count
-------
   110
(1 row)
```

Instead of SELECT *, this query SELECTs only three specific columns: productName, buyPrice (what the company pays the vendor), and MSRP (Manufacturer's Suggested Retail Price, what customers pay). The double quotes around column names are required in PostgreSQL when column names contain uppercase letters or mixed case, as defined in this schema. All 110 products are returned with just these three fields.

9. Get customer names and cities

SELECT "customerName", city FROM customers;

customerName	city
Atelier graphique	Nantes
Signal Gift Stores	Las Vegas
Australian Collectors, Co.	Melbourne
La Rochelle Gifts	Nantes
Baane Mini Imports	Stavern
Mini Gifts Distributors Ltd.	San Rafael
Havel & Zbyszek Co	Warszawa
Blauer See Auto, Co.	Frankfurt
Mini Wheels Co.	San Francisco
Land of Toys Inc.	NYC
Euro+ Shopping Channel	Madrid
Volvo Model Replicas, Co	Luleå
Danish Wholesale Imports	Kobenhavn
Saveley & Henriot, Co.	Lyon
Dragon Souveniers, Ltd.	Singapore
Muscle Machine Inc	NYC

Selects two specific columns from customers: the customer's business name and the city they're located in. This is a projection query (choosing a subset of columns). The city column is lowercase so it doesn't need quotes, while customerName is a mixed case so it does. Returns 122 rows, one per customer.

10. List employee first and last names

SELECT "firstName", "lastName" FROM employees;

```
classicmodels=# select "firstName", "lastName" from employees;
 firstName | lastName
-----------+-----------
 Diane     | Murphy
 Mary      | Patterson
 Jeff      | Firrelli
 William   | Patterson
 Gerard    | Bondur
 Anthony   | Bow
 Leslie    | Jennings
 Leslie    | Thompson
 Julie     | Firrelli
 Steve     | Patterson
 Foon Yue  | Tseng
 George    | Vanauf
 Loui      | Bondur
 Gerard    | Hernandez
 Pamela    | Castillo
 Larry     | Bott
 Barry     | Jones
 Andy      | Fixter
 Peter     | Marsh
 Tom       | King
 Mami      | Nishi
 Yoshimi   | Kato
 Martin    | Gerard
(23 rows)
```

Projects just the name columns from the employees table. Both column names are mixed case so they're wrapped in double quotes. Returns 23 rows, one per employee.

11. Get all order dates

SELECT distinct("orderDate") FROM orders;

```
 orderDate
------------
 2004-12-03
 2004-09-09
 2004-06-30
 2004-01-29
 2004-04-29
 2004-12-15
 2003-07-24
 2003-07-01
 2004-04-09
 2004-04-03
 2005-05-31
 2004-03-11
 2004-09-07
 2004-12-04
```

```
classicmodels=# select count(distinct("orderDate")) from orders;
 count
-------
   265
(1 row)
```

```
classicmodels=# select count(*) from orders;
 count
-------
   326
(1 row)
```

The key word here is DISTINCT, it removes duplicate values and returns only unique entries. Without DISTINCT, you'd get 326 rows (one date per order), but since multiple orders can happen on the same day, many dates repeat. With DISTINCT, only 265 unique dates are returned. The parentheses around orderDate in DISTINCT("orderDate") are optional in PostgreSQL but are written for clarity.

12. Show product vendor list

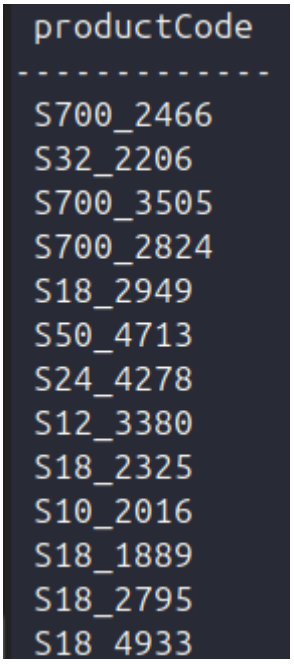
SELECT distinct("productVendor") FROM products;

```
classicmodels=# select distinct("productVendor") from products;
       productVendor
--------------------------
 Welly Diecast Productions
 Motor City Art Classics
 Classic Metal Creations
 Studio M Art Models
 Exoto Designs
 Second Gear Diecast
 Unimax Art Galleries
 Highway 66 Mini Classics
 Autoart Studio Design
 Red Start Diecast
 Carousel DieCast Legends
 Gearbox Collectibles
 Min Lin Diecast
(13 rows)
```

Products can share the same vendor (multiple products from the same manufacturer). Without DISTINCT you'd get 110 rows with many repeating vendor names. DISTINCT collapses all duplicates so only unique vendor names appear. Result: 13 distinct vendors, meaning the company sources products from 13 different manufacturers.

13. Get all product codes

SELECT "productCode" FROM products;



productCode
S700_2466
S32_2206
S700_3505
S700_2824
S18_2949
S50_4713
S24_4278
S12_3380
S18_2325
S10_2016
S18_1889
S18_2795
S18_4933

productCode is the primary key of the products table, meaning every value is already unique by database constraint. So DISTINCT here is redundant hence not used. It returns 110 rows.

14. List all country FROM offices

SELECT distinct(country) FROM offices;

```
classicmodels=# select distinct(country) from offices;
  country
-----------
 USA
 France
 Japan
 UK
 Australia
(5 rows)
```

There are 7 offices but only 5 distinct countries (USA has 3 offices: San Francisco, Boston, NYC). DISTINCT collapses those 3 USA entries into one. The result shows: USA, France, Japan, UK, Australia. This is a good example of DISTINCT being genuinely useful reducing 7 rows to 5 meaningful unique values

15. Show all order statuses

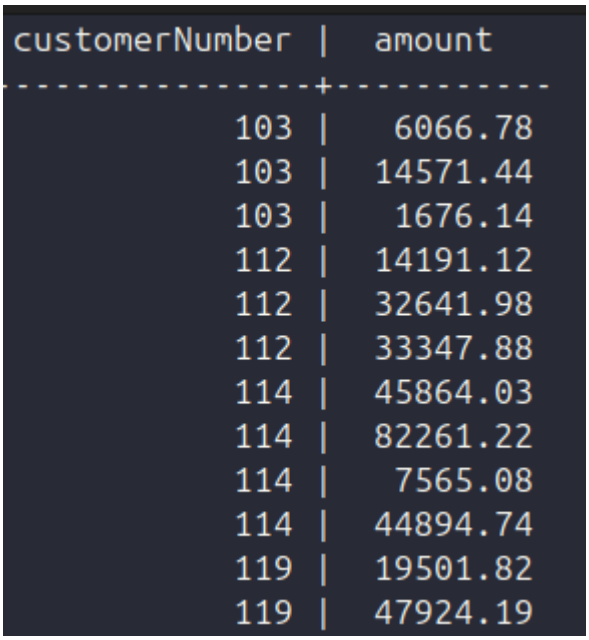
SELECT distinct(status) FROM orders;

```
classicmodels=# select distinct(status) from orders;
   status
------------
 Shipped
 In Process
 Disputed
 Cancelled
 Resolved
 On Hold
(6 rows)
```

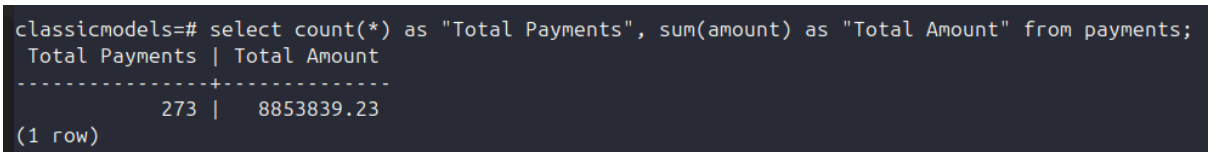
326 orders exist but there are only 6 possible status values used in the system. DISTINCT collapses all repeating statuses to show each unique value once: Shipped, In Process, Disputed, Cancelled, Resolved, On Hold. This is how you discover what values a categorical column contains in a database you're unfamiliar with.

16. Get all payment amounts

SELECT "customerNumber", amount FROM payments;



customerNumber	amount
103	6066.78
103	14571.44
103	1676.14
112	14191.12
112	32641.98
112	33347.88
114	45864.03
114	82261.22
114	7565.08
114	44894.74
119	19501.82
119	47924.19



```
classicmodels=# select count(*) as "Total Payments", sum(amount) as "Total Amount" from payments;
 Total Payments | Total Amount
----------------+--------------
            273 |   8853839.23
(1 row)
```

Projects two columns from payments who paid (customerNumber) and how much (amount). No DISTINCT, so all 273 payments are returned which amount to a total of 8853839.23. A single customer can appear multiple times since they can make multiple payments. For example, customer 103 appears 3 times with amounts 6066.78, 14571.44, and 1676.14.

17. List all job titles

SELECT distinct("jobTitle") FROM employees;

```
classicmodels=# select distinct("jobTitle") from employees;
       jobTitle
----------------------
 VP Sales
 Sales Manager (APAC)
 Sale Manager (EMEA)
 VP Marketing
 Sales Rep
 Sales Manager (NA)
 President
(7 rows)
```

23 employees exist but many share the same job title (most are "Sales Rep"). DISTINCT collapses all duplicates to show only the 7 unique roles in the company: President, VP Sales, VP Marketing, Sales Manager (APAC), Sale Manager (EMEA), Sales Manager (NA), and Sales Rep. This is a useful query for understanding the company's hierarchy.

18. Get customer phone numbers

SELECT "customerNumber", phone FROM customers;

customerNumber	phone
103	40.32.2555
112	7025551838
114	03 9520 4555
119	40.67.8555
121	07-98 9555
124	4155551450
125	(26) 642-7555
128	+49 69 66 90 2555
129	6505555787
131	2125557818
141	(91) 555 94 44
144	0921-12 3555
145	31 12 3555
146	78.32.5555
148	+65 221 7555

Projects customer identifier and their contact phone number. Returns all 122 rows, one per customer. customerNumber is included alongside the phone so you know which number belongs to which customer, since the table has no name in this query. Phone numbers are stored as plain text strings, not numbers, since they contain formatting characters like dashes, spaces, and plus signs.

19. Show product MSRP values

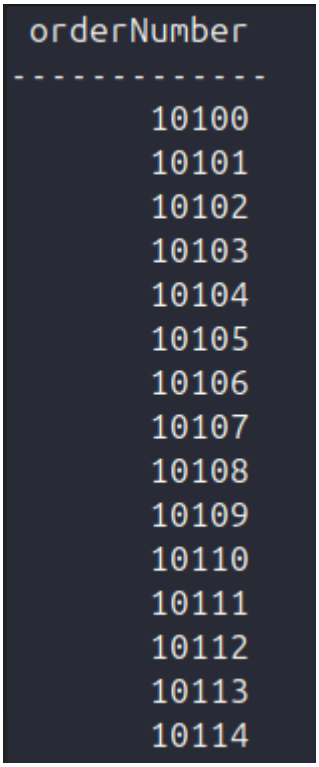
SELECT “productName”, “MSRP” FROM products;

productName	MSRP
1969 Harley Davidson Ultimate Chopper	95.70
1952 Alpine Renault 1300	214.30
1996 Moto Guzzi 1100i	118.94
2003 Harley-Davidson Eagle Drag Bike	193.66
1972 Alfa Romeo GTA	136.00
1962 LanciaA Delta 16V	147.74
1968 Ford Mustang	194.57
2001 Ferrari Enzo	207.80
1958 Setra Bus	136.67
2002 Suzuki XREO	150.62
1969 Corvair Monza	151.08
1968 Dodge Charger	117.44
1969 Ford Falcon	173.02
1970 Plymouth Hemi Cuda	79.80
1957 Chevy Pickup	118.50

Projects the product name and its Manufacturer's Suggested Retail Price. MSRP is a decimal column. Returns all 110 products. Combined with buyPrice, you could calculate profit margins per product but this query only asks for name and MSRP.

20. List order numbers

SELECT "orderNumber" FROM orders;



```
 orderNumber
-------------
       10100
       10101
       10102
       10103
       10104
       10105
       10106
       10107
       10108
       10109
       10110
       10111
       10112
       10113
       10114
```

The most minimal query possible, a single column from one table. Returns all 326 orderNumber values. orderNumber is the primary key so every value is unique. This kind of query is useful when you only need identifiers, for example to loop through orders programmatically.

21. Get orders with customer names

SELECT orders."orderNumber", orders."orderDate", orders."requiredDate", orders."shippedDate", orders.status, customers."customerName" FROM orders JOIN customers ON orders."customerNumber" = customers."customerNumber";

orderNumber	orderDate	requiredDate	shippedDate	status	customerName
10100	2003-01-06	2003-01-13	2003-01-10	Shipped	Online Diecast Creations Co.
10101	2003-01-09	2003-01-18	2003-01-11	Shipped	Blauer See Auto, Co.
10102	2003-01-10	2003-01-18	2003-01-14	Shipped	Vitachrome Inc.
10103	2003-01-29	2003-02-07	2003-02-02	Shipped	Baane Mini Imports
10104	2003-01-31	2003-02-09	2003-02-01	Shipped	Euro+ Shopping Channel
10105	2003-02-11	2003-02-21	2003-02-12	Shipped	Danish Wholesale Imports
10106	2003-02-17	2003-02-24	2003-02-21	Shipped	Rovelli Gifts
10107	2003-02-24	2003-03-03	2003-02-26	Shipped	Land of Toys Inc.
10108	2003-03-03	2003-03-12	2003-03-08	Shipped	Cruz & Sons Co.
10109	2003-03-10	2003-03-19	2003-03-11	Shipped	Motor Mint Distributors Inc.
10110	2003-03-18	2003-03-24	2003-03-20	Shipped	AV Stores, Co.
10111	2003-03-25	2003-03-31	2003-03-30	Shipped	Mini Wheels Co.
10112	2003-03-24	2003-04-03	2003-03-29	Shipped	Volvo Model Replicas, Co
10113	2003-03-26	2003-04-02	2003-03-27	Shipped	Mini Gifts Distributors Ltd.
10114	2003-04-01	2003-04-07	2003-04-02	Shipped	La Corne Dabondance, Co.

```
classicmodels=# select count(*) from orders join customers on orders."customerNumber" = customers."customerNumber";
 count
-------
   326
(1 row)
```

This is the JOIN query. The orders table stores customerNumber as a foreign key, just a number. To get the actual customer name, you must JOIN the customers table. The JOIN condition orders.customerNumber = customers.customerNumber is the bridge which tells PostgreSQL to match each order row with the customer row that has the same customerNumber. The result combines columns from both tables into one output row. All 326 orders appear because every order has a valid customer, so the INNER JOIN matches all rows.

22. Get employees with office city

SELECT e."employeeNumber", e."firstName", e."lastName", e."reportsTo", e."jobTitle", e."officeCode", o.city FROM employees e JOIN offices o ON e."officeCode" = o."officeCode";

```
classicmodels=# select e."employeeNumber", e."firstName", e."lastName",e."reportsTo",e."jobTitle",e."officeCode",o.city from empl
oyees e join offices o on e."officeCode" = o."officeCode";
 employeeNumber | firstName | lastName  | reportsTo |       jobTitle       | officeCode |     city
----------------+-----------+-----------+-----------+----------------------+------------+---------------
           1002 | Diane     | Murphy    |           | President            | 1          | San Francisco
           1056 | Mary      | Patterson |      1002 | VP Sales             | 1          | San Francisco
           1076 | Jeff      | Firrelli  |      1002 | VP Marketing         | 1          | San Francisco
           1088 | William   | Patterson |      1056 | Sales Manager (APAC) | 6          | Sydney
           1102 | Gerard    | Bondur    |      1056 | Sale Manager (EMEA)  | 4          | Paris
           1143 | Anthony   | Bow       |      1056 | Sales Manager (NA)   | 1          | San Francisco
           1165 | Leslie    | Jennings  |      1143 | Sales Rep            | 1          | San Francisco
           1166 | Leslie    | Thompson  |      1143 | Sales Rep            | 1          | San Francisco
           1188 | Julie     | Firrelli  |      1143 | Sales Rep            | 2          | Boston
           1216 | Steve     | Patterson |      1143 | Sales Rep            | 2          | Boston
           1286 | Foon Yue  | Tseng     |      1143 | Sales Rep            | 3          | NYC
           1323 | George    | Vanauf    |      1143 | Sales Rep            | 3          | NYC
           1337 | Loui      | Bondur    |      1102 | Sales Rep            | 4          | Paris
           1370 | Gerard    | Hernandez |      1102 | Sales Rep            | 4          | Paris
           1401 | Pamela    | Castillo  |      1102 | Sales Rep            | 4          | Paris
           1501 | Larry     | Bott      |      1102 | Sales Rep            | 7          | London
           1504 | Barry     | Jones     |      1102 | Sales Rep            | 7          | London
           1611 | Andy      | Fixter    |      1088 | Sales Rep            | 6          | Sydney
           1612 | Peter     | Marsh     |      1088 | Sales Rep            | 6          | Sydney
           1619 | Tom       | King      |      1088 | Sales Rep            | 6          | Sydney
           1621 | Mami      | Nishi     |      1056 | Sales Rep            | 5          | Tokyo
           1625 | Yoshimi   | Kato      |      1621 | Sales Rep            | 5          | Tokyo
           1702 | Martin    | Gerard    |      1102 | Sales Rep            | 4          | Paris
(23 rows)
```

JOIN between employees and offices via the officeCode foreign key. The aliases e and o are shorthand, instead of writing employees.columnName every time, you write e.columnName. The JOIN condition matches each employee to the office they belong to. o.city brings in the city from the offices table. Returns 23 rows — all employees with their office city added.

23. Get payments with customer names

SELECT p."checkNumber", p."paymentDate", p.amount, c."customerName" FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";

```
classicmodels=# select count(*) from payments p join customers c on p."customerNumber" = c."customerNumber";
 count
-------
   273
(1 row)
```

checkNumber	paymentDate	amount	customerName
HQ336336	2004-10-19	6066.78	Atelier graphique
JM555205	2003-06-05	14571.44	Atelier graphique
OM314933	2004-12-18	1676.14	Atelier graphique
BO864823	2004-12-17	14191.12	Signal Gift Stores
HQ55022	2003-06-06	32641.98	Signal Gift Stores
ND748579	2004-08-20	33347.88	Signal Gift Stores
GG31455	2003-05-20	45864.03	Australian Collectors, Co.
MA765515	2004-12-15	82261.22	Australian Collectors, Co.
NP603840	2003-05-31	7565.08	Australian Collectors, Co.
NR27552	2004-03-10	44894.74	Australian Collectors, Co.
DB933704	2004-11-14	19501.82	La Rochelle Gifts
LN373447	2004-08-08	47924.19	La Rochelle Gifts
NG94694	2005-02-22	49523.67	La Rochelle Gifts

JOIN between payments and customers on customerNumber. p is the alias for payments, c for customers. The payments table only stores the customerNumber foreign key, the actual name lives in customers. The JOIN fetches the name and attaches it to each payment row. Returns 273 rows, all payments enriched with who made them.

24. Get order details with product names

SELECT od."orderNumber", od."quantityOrdered", od."priceEach", od."orderLineNumber", p."productName" FROM orderdetails od JOIN products p ON od."productCode" = p."productCode";

orderNumber	quantityOrdered	priceEach	orderLineNumber	productName
10100	30	136.00	3	1917 Grand Touring Sedan
10100	50	55.09	2	1911 Ford Town Car
10100	22	75.46	4	1932 Alfa Romeo 8C2300 Spider Sport
10100	49	35.29	1	1936 Mercedes Benz 500k Roadster
10101	25	108.06	4	1932 Model A Ford J-Coupe
10101	26	167.06	1	1928 Mercedes-Benz SSK
10101	45	32.53	3	1939 Chevrolet Deluxe Coupe
10101	46	44.35	2	1938 Cadillac V-16 Presidential Limousine
10102	39	95.55	2	1937 Lincoln Berline
10102	41	43.13	1	1936 Mercedes-Benz 500K Special Roadster
10103	26	214.30	11	1952 Alpine Renault 1300
10103	42	119.67	4	1962 LanciaA Delta 16V
10103	27	121.64	8	1958 Setra Bus
10103	35	94.50	10	1940 Ford Pickup Truck
10103	22	58.34	2	1926 Ford Fire Engine

```
classicmodels=# select count(*) from orderdetails od join products p on od."productCode" = p."productCode";
 count
-------
  2996
(1 row)
```

JOIN between orderdetails and products on productCode. The orderdetails table has one row per product per order (an order can contain multiple products, each on its own line). The productCode foreign key links each line to the product's full details. p.productName brings in the human readable product name. Returns 2,996 rows.

25. Get products with product line description

SELECT p."productName", pl."productLine", pl."textDescription", pl."htmlDescription" FROM products p JOIN productlines pl ON p."productLine" = pl."productLine";

```
classicmodels=# select count(*) from products p join productlines pl on p."productLine" = pl."productLine";
 count
-------
   110
(1 row)
```

```
               productName                |   productLine    |

                                textDescription

                          | htmlDescription
-----------------------------------------+------------------+-----------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
--------------------------+-----------------
 1969 Harley Davidson Ultimate Chopper       | Motorcycles      | Our motorcycles are state of the art replicas of classic as wel
l as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models contain stunning details such as official
logos, rotating wheels, working kickstand, front suspension, gear-shift lever, footbrake lever, and drive chain. Materials used i
nclude diecast and plastic. The models range in size from 1:10 to 1:50 scale and include numerous limited edition and several out
-of-production vehicles. All models come fully assembled and ready for display in the home or office. Most include a certificate
of authenticity.
                          |
 1952 Alpine Renault 1300                   | Classic Cars     | Attention car enthusiasts: Make your wildest car ownership drea
ms come true. Whether you are looking for classic muscle cars, dream sports cars or movie-inspired miniatures, you will find grea
t choices in this category. These replicas feature superb attention to detail and craftsmanship and offer features such as workin
g steering system, opening forward compartment, opening rear trunk with removable spare wheel, 4-wheel independent spring suspens
ion, and so on. The models range in size from 1:10 to 1:24 scale and include numerous limited edition and several out-of-producti
on vehicles. All models include a certificate of authenticity from their manufacturers and come fully assembled and ready for dis
play in the home or office. |
```

JOIN between products and productlines on productLine. The products table stores productLine as a foreign key (a short string like "Motorcycles"). The productlines table holds the detailed description for each category. The JOIN attaches the full textDescription and htmlDescription to each product. Returns 110 rows, all products with their category description appended.

26. Get customers with sales rep names

SELECT c."customerNumber", c."customerName", e."firstName", e."lastName" FROM customers c JOIN employees e ON c."salesRepEmployeeNumber" = e."employeeNumber";

customerNumber	customerName	firstName	lastName
103	Atelier graphique	Gerard	Hernandez
112	Signal Gift Stores	Leslie	Thompson
114	Australian Collectors, Co.	Andy	Fixter
119	La Rochelle Gifts	Gerard	Hernandez
121	Baane Mini Imports	Barry	Jones
124	Mini Gifts Distributors Ltd.	Leslie	Jennings
128	Blauer See Auto, Co.	Barry	Jones
129	Mini Wheels Co.	Leslie	Jennings
131	Land of Toys Inc.	George	Vanauf
141	Euro+ Shopping Channel	Gerard	Hernandez
144	Volvo Model Replicas, Co	Barry	Jones
145	Danish Wholesale Imports	Pamela	Castillo
146	Saveley & Henriot, Co.	Loui	Bondur
148	Dragon Souveniers, Ltd.	Mami	Nishi
151	Muscle Machine Inc	Foon Yue	Tseng
157	Diecast Classics Inc.	Steve	Patterson

This JOIN uses foreign key, salesRepEmployeeNumber in the customers table which references employeeNumber in the employees table. The column names are different on each side, which is why the ON condition explicitly spells out both sides. This links each customer to the employee who manages their account. Returns 100 rows (not 122) because 22 customers have no assigned sales rep (NULL in salesRepEmployeeNumber), and INNER JOIN excludes those non matching rows.

27. Get orders with customer city

SELECT o."orderNumber", o."orderDate", o."requiredDate",o."shippedDate", o.status, c.city FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";

orderNumber	orderDate	requiredDate	shippedDate	status	city
10100	2003-01-06	2003-01-13	2003-01-10	Shipped	Nashua
10101	2003-01-09	2003-01-18	2003-01-11	Shipped	Frankfurt
10102	2003-01-10	2003-01-18	2003-01-14	Shipped	NYC
10103	2003-01-29	2003-02-07	2003-02-02	Shipped	Stavern
10104	2003-01-31	2003-02-09	2003-02-01	Shipped	Madrid
10105	2003-02-11	2003-02-21	2003-02-12	Shipped	Kobenhavn
10106	2003-02-17	2003-02-24	2003-02-21	Shipped	Bergamo
10107	2003-02-24	2003-03-03	2003-02-26	Shipped	NYC
10108	2003-03-03	2003-03-12	2003-03-08	Shipped	Makati City
10109	2003-03-10	2003-03-19	2003-03-11	Shipped	Philadelphia
10110	2003-03-18	2003-03-24	2003-03-20	Shipped	Manchester
10111	2003-03-25	2003-03-31	2003-03-30	Shipped	San Francisco
10112	2003-03-24	2003-04-03	2003-03-29	Shipped	Luleå
10113	2003-03-26	2003-04-02	2003-03-27	Shipped	San Rafael
10114	2003-04-01	2003-04-07	2003-04-02	Shipped	Paris
10115	2003-04-04	2003-04-12	2003-04-07	Shipped	NYC

JOIN between orders and customers on customerNumber. o is the alias for orders, c for customers. Returns 326 rows. This allows you to see where (geographically) each order originated.

28. Get employees and their manager

SELECT e."employeeNumber",e."firstName" AS "Emp_FirstName", e."lastName" AS "Emp_LastName", m."firstName" AS "Manager_FirstName", m."lastName" AS "Manager_LastName" FROM employees e LEFT JOIN employees m ON e."reportsTo" = m."employeeNumber";

```
classicmodels=# select e."employeeNumber",e."firstName" as "Emp_FirstName", e."lastName" as "Emp_LastName", m."firstName" as "Man
ager_FirstName", m."lastName" as "Manager_LastName" from employees e left join employees m on e."reportsTo" = m."employeeNumber";
 employeeNumber | Emp_FirstName | Emp_LastName | Manager_FirstName | Manager_LastName
----------------+---------------+--------------+-------------------+-----------------
           1002 | Diane         | Murphy       |                   |
           1056 | Mary          | Patterson    | Diane             | Murphy
           1076 | Jeff          | Firrelli     | Diane             | Murphy
           1088 | William       | Patterson    | Mary              | Patterson
           1102 | Gerard        | Bondur       | Mary              | Patterson
           1143 | Anthony       | Bow          | Mary              | Patterson
           1165 | Leslie        | Jennings     | Anthony           | Bow
           1166 | Leslie        | Thompson     | Anthony           | Bow
           1188 | Julie         | Firrelli     | Anthony           | Bow
           1216 | Steve         | Patterson    | Anthony           | Bow
           1286 | Foon Yue      | Tseng        | Anthony           | Bow
           1323 | George        | Vanauf       | Anthony           | Bow
           1337 | Loui          | Bondur       | Gerard            | Bondur
           1370 | Gerard        | Hernandez    | Gerard            | Bondur
           1401 | Pamela        | Castillo     | Gerard            | Bondur
           1501 | Larry         | Bott         | Gerard            | Bondur
           1504 | Barry         | Jones        | Gerard            | Bondur
           1611 | Andy          | Fixter       | William           | Patterson
           1612 | Peter         | Marsh        | William           | Patterson
           1619 | Tom           | King         | William           | Patterson
           1621 | Mami          | Nishi        | Mary              | Patterson
           1625 | Yoshimi       | Kato         | Mami              | Nishi
           1702 | Martin        | Gerard       | Gerard            | Bondur
(23 rows)
```

This is a self join, the employees table is joined to itself. The table is aliased twice: e represents the employee, m represents their manager (who is also an employee in the same table). The join condition e.reportsTo = m.employeeNumber matches each employee's manager ID to the corresponding employee row. LEFT JOIN is critical here as it ensures that the President (who has no manager, so reportsTo is NULL) is still included in the results with NULL values in the manager columns. An INNER JOIN would exclude her. Returns all 23 employees.

29. Get orderdetails with product vendor

SELECT od."orderNumber", od."productCode", od."quantityOrdered", od."priceEach", od."orderLineNumber", p."productVendor" FROM orderdetails od JOIN products p ON od."productCode" = p."productCode";

orderNumber	productCode	quantityOrdered	priceEach	orderLineNumber	productVendor
10100	S18_1749	30	136.00	3	Welly Diecast Productions
10100	S18_2248	50	55.09	2	Motor City Art Classics
10100	S18_4409	22	75.46	4	Exoto Designs
10100	S24_3969	49	35.29	1	Red Start Diecast
10101	S18_2325	25	108.06	4	Autoart Studio Design
10101	S18_2795	26	167.06	1	Gearbox Collectibles
10101	S24_1937	45	32.53	3	Motor City Art Classics
10101	S24_2022	46	44.35	2	Classic Metal Creations
10102	S18_1342	39	95.55	2	Motor City Art Classics
10102	S18_1367	41	43.13	1	Studio M Art Models
10103	S10_1949	26	214.30	11	Classic Metal Creations
10103	S10_4962	42	119.67	4	Second Gear Diecast
10103	S12_1666	27	121.64	8	Welly Diecast Productions
10103	S18_1097	35	94.50	10	Studio M Art Models
10103	S18_2432	22	58.34	2	Carousel DieCast Legends

JOIN between orderdetails and products on productCode. This tells you which manufacturer/vendor supplied the product in each order line. Returns 2,996 rows, all order line items along with the vendor who made that product.

30. Get payments with customer country

SELECT p."customerNumber", p."paymentDate", p.amount, c.country FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";

customerNumber	paymentDate	amount	country
103	2004-10-19	6066.78	France
103	2003-06-05	14571.44	France
103	2004-12-18	1676.14	France
112	2004-12-17	14191.12	USA
112	2003-06-06	32641.98	USA
112	2004-08-20	33347.88	USA
114	2003-05-20	45864.03	Australia
114	2004-12-15	82261.22	Australia
114	2003-05-31	7565.08	Australia
114	2004-03-10	44894.74	Australia
119	2004-11-14	19501.82	France
119	2004-08-08	47924.19	France
119	2005-02-22	49523.67	France
121	2003-02-16	50218.95	Norway
121	2003-10-28	1491.38	Norway
121	2004-11-04	17876.32	Norway

JOIN between payments and customers on customerNumber, pulling c.country from the customers table. This enriches each payment with the country of origin of the paying customer. Returns 273 rows. Useful for geographic revenue analysis.

31. Count customers per country

SELECT country, COUNT(*) AS "customers_COUNT" FROM customers GROUP BY country;

```
classicmodels=# select country, count(*) as "customers_count" from customers group by country;
   country    | customers_count
--------------+-----------------
 Switzerland  |               3
 New Zealand  |               4
 Italy        |               4
 Russia       |               1
 Sweden       |               2
 Norway       |               1
 Norway       |               2
 USA          |              36
 Netherlands  |               1
 Austria      |               2
 UK           |               5
 Australia    |               5
 Ireland      |               2
 Germany      |              13
 Singapore    |               3
 Canada       |               3
 Finland      |               3
 Portugal     |               2
 Spain        |               7
 Belgium      |               2
 France       |              12
 Israel       |               1
 South Africa |               1
 Poland       |               1
 Hong Kong    |               1
 Japan        |               2
 Denmark      |               2
 Philippines  |               1
(28 rows)
```

This is the aggregation query using GROUP BY. GROUP BY country splits all 122 customer rows into groups: one group per unique country value. COUNT(*) then counts how many rows are in each group. The result has one row per country (28 countries total), with the count of customers in that country.

32. Total payments per customer

SELECT p."customerNumber", c."customerName", c.phone, c.city, c.state, c.country, SUM(p.amount) AS "totalPayments" FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber" GROUP BY p."customerNumber", c."customerName", c.phone, c.city, c.state, c.country;

customerNumber	customerName	phone	city	state	country	totalPayments
103	Atelier graphique	40.32.2555	Nantes		France	22314.36
350	Marseille Mini Autos	91.24.4555	Marseille		France	71547.53
181	Vitachrome Inc.	2125551500	NYC	NY	USA	72497.64
495	Diecast Collectables	6175552555	Boston	MA	USA	65541.74
250	Lyon Souveniers	+33 1 46 62 7555	Paris		France	67659.19
121	Baane Mini Imports	07-98 9555	Stavern		Norway	104224.79
448	Scandinavian Gift Ideas	0695-34 6555	Bräcke		Sweden	76776.44
172	La Corne Dabondance, Co.	(1) 42.34.2555	Paris		France	86553.52
216	Enaco Distributors	(93) 203 4555	Barcelona		Spain	68520.47
339	Classic Gift Ideas, Inc	2155554695	Philadelphia	PA	USA	57939.34
151	Muscle Machine Inc	2125557413	NYC	NY	USA	177913.95
406	Auto Canal+ Petit	(1) 47.55.6555	Paris		France	86436.97
471	Australian Collectables, Ltd	61-9-3844-6555	Glen Waverly	Victoria	Australia	44920.76
334	Suominen Souveniers	+358 9 8045 555	Espoo		Finland	103896.74
209	Mini Caravy	88.60.1555	Strasbourg		France	75859.32
321	Corporate Gift Ideas Co.	6505551386	San Francisco	CA	USA	132340.78
484	Iberia Gift Imports, Corp.	(95) 555 82 82	Sevilla		Spain	50987.85
299	Norway Gifts By Mail, Co.	+47 2212 1555	Oslo		Norway	69059.04

```
classicmodels=# select sum(amount) from payments where "customerNumber" = 103;
   sum
----------
 22314.36
(1 row)

classicmodels=# select sum(amount) from payments where "customerNumber" = 350;
   sum
----------
 71547.53
(1 row)
```

Combines JOIN and GROUP BY together. First, the JOIN links each payment to its customer (getting name, phone, city, etc.). Then GROUP BY collapses all payment rows for the same customer into a single summary row. SUM(p.amount) adds up all payments that customer has ever made. The GROUP BY must include every non-aggregated column in the SELECT list, this is a PostgreSQL requirement. Returns 98 rows (customers who have made at least one payment).

33. Number of orders per status

SELECT status, COUNT(*) FROM orders GROUP BY status;

```
classicmodels=# select status, count(*) from orders group by status;
   status   | count
------------+-------
 Shipped    |   303
 In Process |     6
 Disputed   |     3
 Cancelled  |     6
 Resolved   |     4
 On Hold    |     4
(6 rows)
```

Groups all 326 order rows by their status value, then counts how many orders fall into each group. Result: 6 rows, one per status. This reveals the distribution of orders: Shipped=303 (92.9%), In Process=6, Cancelled=6, Resolved=4, On Hold=4, Disputed=3. A single-table aggregation, no JOIN needed.

34. Products per product line

SELECT "productLine", COUNT(*) AS "productCount" FROM products GROUP BY "productLine";

```
classicmodels=# select "productLine", count(*) as "productCount" from products group by "productLine";
   productLine    | productCount
------------------+--------------
 Classic Cars     |           38
 Trains           |            3
 Planes           |           12
 Trucks and Buses |           11
 Vintage Cars     |           24
 Motorcycles      |           13
 Ships            |            9
(7 rows)
```

Groups all 110 product rows by their productLine category, then counts products per group. Returns 7 rows, one per product line. The double quotes around productLine are needed due to the mixed case. Result: Classic Cars=38 (most), Trains=3 (fewest). No JOIN needed because productLine is a column that already exists in the products table (as a foreign key stored directly as a string).

35. Employees per office

SELECT o."officeCode", o.city,o.state, o.country, COUNT(*) AS "Employee_COUNT" FROM employees e JOIN offices o ON e."officeCode" = o."officeCode" GROUP BY o."officeCode", o.city, o.state, o.country;

```
classicmodels=# select o."officeCode", o.city,o.state, o.country, count(*) as "E
mployee_count" from employees e join offices o on e."officeCode" = o."officeCode
" group by o."officeCode", o.city, o.state, o.country;
 officeCode |     city      |   state    |  country  | Employee_count
------------+---------------+------------+-----------+----------------
 6          | Sydney        |            | Australia |              4
 2          | Boston        | MA         | USA       |              2
 4          | Paris         |            | France    |              5
 7          | London        |            | UK        |              2
 5          | Tokyo         | Chiyoda-Ku | Japan     |              2
 1          | San Francisco | CA         | USA       |              6
 3          | NYC           | NY         | USA       |              2
(7 rows)
```

This query combines a JOIN and GROUP BY aggregation to count how many employees work at each office. The employees table stores officeCode as a foreign key while the offices table holds the actual city, state, and country details for each office. The JOIN condition e."officeCode" = o."officeCode" bridges the two tables, attaching the office's location details to each employee row. Then GROUP BY o."officeCode", o.city, o.state, o.country collapses all employees belonging to the same office into a single group, and COUNT(*) counts how many employees were in each group. The GROUP BY must include all non-aggregated columns in the SELECT list: officeCode, city, state, country, as required by PostgreSQL. Returns 7 rows, one per office.

36. Total stock per product vendor

SELECT "productVendor", SUM("quantityInStock") AS "Total_Stock" FROM products GROUP BY "productVendor";

```
classicmodels=# select "productVendor", sum("quantityInStock") as "Total_Stock" from products group by "productVendor";
       productVendor       | Total_Stock
---------------------------+-------------
 Welly Diecast Productions |       45095
 Motor City Art Classics   |       43105
 Classic Metal Creations   |       45408
 Studio M Art Models       |       42253
 Exoto Designs             |       44166
 Second Gear Diecast       |       42865
 Unimax Art Galleries      |       38191
 Highway 66 Mini Classics  |       37520
 Autoart Studio Design     |       30093
 Red Start Diecast         |       35046
 Carousel DieCast Legends  |       40805
 Gearbox Collectibles      |       60495
 Min Lin Diecast           |       50089
(13 rows)
```

Groups all 110 products by their vendor, then uses SUM(quantityInStock) to total the stock units across all products that vendor supplies. Returns 13 rows, one per vendor. This is useful for understanding supplier dependency.

37. Average buy price per product line

SELECT "productLine", avg("buyPrice") AS "Average_Buy_Price" FROM products GROUP BY "productLine";

```
classicmodels=# select "productLine", avg("buyPrice") as "Average_Buy_Price" from products group by "productLine";
   productLine    |  Average_Buy_Price
------------------+--------------------
 Classic Cars     | 64.4463157894736842
 Trains           | 43.9233333333333333
 Planes           | 49.6291666666666667
 Trucks and Buses | 56.3290909090909091
 Vintage Cars     | 46.0662500000000000
 Motorcycles      | 50.6853846153846154
 Ships            | 47.0077777777777778
(7 rows)
```

```
classicmodels=# select avg("buyPrice") from products where "productLine" = 'Classic Cars';
         avg
---------------------
 64.4463157894736842
(1 row)

classicmodels=# select avg("buyPrice") from products where "productLine" = 'Trains';
         avg
---------------------
 43.9233333333333333
(1 row)
```

Groups products by productLine and calculates the mean buyPrice within each group using AVG(). Returns 7 rows. The result values have many decimal places (e.g., 64.4463157894736842 for Classic Cars) because PostgreSQL's AVG returns full precision. Classic Cars have the highest average buy price, Trains the lowest.

38. Orders per customer

SELECT o."customerNumber", c."customerName", c.phone, c.city, c.country, COUNT(*) AS "totalOrders" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber" GROUP BY o."customerNumber", c."customerName", c.phone, c.city, c.country;

customerNumber	customerName	phone	city	country	totalOrders
148	Dragon Souveniers, Ltd.	+65 221 7555	Singapore	Singapore	5
175	Gift Depot Inc.	2035552570	Bridgewater	USA	3
314	Petit Auto	(02) 5554 67	Bruxelles	Belgium	3
347	Men'R' US Retailers, Ltd.	2155554369	Los Angeles	USA	2
181	Vitachrome Inc.	2125551500	NYC	USA	3
471	Australian Collectables, Ltd	61-9-3844-6555	Glen Waverly	Australia	3
344	CAF Imports	+34 913 728 555	Madrid	Spain	2
173	Cambridge Collectables Co.	6175555555	Cambridge	USA	2
204	Online Mini Collectables	6175557555	Brickhaven	USA	2
487	Signal Collectibles Ltd.	4155554312	Brisbane	USA	2
119	La Rochelle Gifts	40.67.8555	Nantes	France	4
339	Classic Gift Ideas, Inc	2155554695	Philadelphia	USA	2
249	Amica Models & Co.	011-4988555	Torino	Italy	2
128	Blauer See Auto, Co.	+49 69 66 90 2555	Frankfurt	Germany	4

```
classicmodels=# select count(*) from orders where "customerNumber" = 148;
 count
-------
     5
(1 row)

classicmodels=# select count(*) from orders where "customerNumber" = 175;
 count
-------
     3
(1 row)
```

JOIN + GROUP BY pattern, counting orders per customers The JOIN brings in customer details, then GROUP BY collapses all orders for the same customer into one row, with COUNT(*) counting how many orders that customer placed. Returns 98 rows, customers who have placed at least one order.

39. Max MSRP per product line

SELECT "productLine", MAX("MSRP") AS "Max_MSRP" FROM products GROUP BY "productLine";

```
classicmodels=# select max("MSRP") from products where "productLine" = 'Trains';
  max
--------
 100.84
(1 row)

classicmodels=# select max("MSRP") from products where "productLine" = 'Ships';
  max
--------
 122.89
(1 row)
```

```
classicmodels=# select "productLine", max("MSRP") as "Max_MSRP" from products group by "productLine";
   productLine    | Max_MSRP
------------------+----------
 Classic Cars     |   214.30
 Trains           |   100.84
 Planes           |   157.69
 Trucks and Buses |   136.67
 Vintage Cars     |   170.00
 Motorcycles      |   193.66
 Ships            |   122.89
(7 rows)
```

Groups products by line and finds the highest MSRP within each group using MAX(). Returns 7 rows showing the most expensive product in each category. Classic Cars has the highest at $214.30 (the 1952 Alpine Renault 1300), Trains the lowest maximum at $100.84. MAX() scans all values in the group and returns only the largest.

40. Min buy price per vendor

SELECT "productVendor", min("buyPrice") AS "Min_Buy_Price" FROM products GROUP BY "productVendor";

```
classicmodels=# select min("buyPrice") from products where "productVendor" = 'Welly Diecast Productions';
  min
-------
 24.23
(1 row)

classicmodels=# select min("buyPrice") from products where "productVendor" = 'Motor City Art Classics';
  min
-------
 22.57
(1 row)
```

```
classicmodels=# select "productVendor", min("buyPrice") as "Min_Buy_Price" from products group by "productVendor";
       productVendor       | Min_Buy_Price
---------------------------+---------------
 Welly Diecast Productions |         24.23
 Motor City Art Classics   |         22.57
 Classic Metal Creations   |         20.61
 Studio M Art Models       |         23.14
 Exoto Designs             |         43.26
 Second Gear Diecast       |         16.24
 Unimax Art Galleries      |         33.30
 Highway 66 Mini Classics  |         32.37
 Autoart Studio Design     |         26.30
 Red Start Diecast         |         21.75
 Carousel DieCast Legends  |         15.91
 Gearbox Collectibles      |         24.14
 Min Lin Diecast           |         34.35
(13 rows)
```

Groups products by vendor and finds the cheapest buyPrice in each vendor's product range using MIN(). Returns 13 rows, one per vendor. Carousel DieCast Legends has the lowest minimum at $15.91, meaning their cheapest product costs the company just $15.91 to acquire. MIN() is the counterpart to MAX(), it scans the group and returns the smallest value.

41. Total number of customers

SELECT COUNT(*) AS "Total Customers" FROM customers;

```
classicmodels=# SELECT COUNT(*) AS "Total Customers" FROM customers;
 Total Customers
-----------------
             122
(1 row)
```

COUNT(*) counts how many unique customerNumber values exist. Since customerNumber is the primary key, every value is already unique, so the result is simply 122, matching the total row count.

42. Total number of products

SELECT COUNT(*) AS "Total Products" FROM products;

```
classicmodels=# SELECT COUNT(*) AS "Total Products" FROM products;
 Total Products
----------------
            110
(1 row)
```

COUNT(*) counts unique product codes. Since productCode is the primary key, all 110 are unique and the result is 110.

43. Total revenue FROM payments

SELECT SUM(amount) AS "Total Revenue" FROM payments;

```
classicmodels=# select sum(amount) as "Total Revenue" from payments;
 Total Revenue
---------------
    8853839.23
(1 row)
```

SUM(amount) is an aggregate function that adds up the amount column across all 273 payment rows in the table. No GROUP BY is needed here. Without it, the aggregation collapses all rows into a single summary value. The result is $8,853,839.23, the total of all payments ever recorded in the system.

44. Average product price

SELECT avg("buyPrice") AS "Avg Product Price" FROM products;

```
classicmodels=# SELECT avg("buyPrice") AS "Avg Product Price" FROM products;
  Avg Product Price
---------------------
 54.3951818181818182
(1 row)
```

AVG(buyPrice) calculates the arithmetic mean of all 110 buyPrice values, adds them all up and divides by 110. No GROUP BY means it aggregates across the entire table into one value. Result: $54.39 (full precision: 54.395181818...). This is a global average across all product lines combined.

45. Max payment amount

SELECT MAX(amount) AS "Max Payment Amount" FROM payments;

```
classicmodels=# select max(amount) as "Max Payment Amount" from payments;
 Max Payment Amount
--------------------
          120166.58
(1 row)
```

MAX(amount) scans all 273 payment amounts and returns only the single largest value: $120,166.58. No GROUP BY means it finds the maximum across the entire payments table. This represents the largest single payment transaction ever recorded.

46. Min payment amount

SELECT min(amount) AS "Min Payment Amount" FROM payments;

```
classicmodels=# select min(amount) as "Min Payment Amount" from payments;
 Min Payment Amount
--------------------
             615.45
(1 row)
```

MIN(amount) is the counterpart to MAX, it scans all 273 values and returns only the smallest: $615.45. This is the minimum single payment in the system. Together with MAX ($120,166.58) and AVG, MIN gives a full picture of the payment distribution range.

47. Count total orders

SELECT COUNT(*) AS "Total Orders" FROM orders;

```
classicmodels=# select count(*) as "Total Orders" from orders;
 Total Orders
--------------
          326
(1 row)
```

COUNT(*) counts every row in the orders table regardless of column values. The * means "count all rows, including any with NULL values in some columns." Returns 326, the total number of orders ever placed in the system. This is the simplest possible aggregation query.

48. Total quantity in stock

SELECT SUM("quantityInStock") AS "Total Quantity in Stocks" FROM products;

```
classicmodels=# select sum("quantityInStock") as "Total Quantity in Stocks" from products;
 Total Quantity in Stocks
--------------------------
                   555131
(1 row)
```

SUM(quantityInStock) adds up the stock level of all 110 products in the catalog. Returns 555,131, meaning the company has over half a million individual product units in their warehouses across all products combined. No GROUP BY means it collapses everything into one total figure.

49. Average MSRP

SELECT avg("MSRP") AS "Average MSRP" FROM products;

```
classicmodels=# select  avg("MSRP") as "Average MSRP" from products;
     Average MSRP
----------------------
 100.4387272727272727
(1 row)
```

AVG(MSRP) calculates the arithmetic mean of all 110 MSRP values, adds them all up and divides by 110. No GROUP BY means it aggregates across the entire table into one value. Result: $100.44 (full precision: 100.4387272727...). This is a global average across all product lines combined.

50. Number of employees

SELECT COUNT(*) AS "Total Employees" FROM employees;

```
classicmodels=# select count(*) as "Total Employees" from employees;
 Total Employees
-----------------
              23
(1 row)
```

COUNT(*) on the employees table counts every row and returns 23, the total number of people working at the company. No filter, no join, no GROUP BY, just a single aggregate value representing the company headcount.